

Exam B - School Solution

Question 1

Since $a^i \sim_L a^{i+j}$, we have that for every $z \in \Sigma^*$, $a^i z \in L \iff a^{i+j} z \in L$. Applying this to z 's of the form $z = a^j z'$, we have

$$\forall z' \in \Sigma^*, \quad a^{i+j} z' \in L \iff a^{i+2j} z' \in L.$$

This implies that $a^{i+j} \sim_L a^{i+2j}$, by the definition of Myhill-Nerode equivalence, and by the transitivity of the equivalence relation we also get $a^i \sim_L a^{i+2j}$ as desired.

Question 2

Let A be an equivalence class. If it contains only one word, we are done. Otherwise, suppose A contains more than one word. Then there exist integers $0 \leq i < j < \infty$ such that $a^i, a^j \in A$. In particular, since $a^i \sim a^j$, it follows that

$$a^i \sim a^{i+(j-i)}.$$

From Question 1, we know (by induction) that for all $k \in \mathbb{N}$,

$$a^i \sim a^{i+k(j-i)}.$$

Hence, for all $k \in \mathbb{N} \cup \{0\}$, we have $a^{i+k(j-i)} \in A$, so the equivalence class A contains infinitely many distinct words as $j - i > 0$.

Question 3

The claim is **correct**.

Assume L is regular. Then there exists a DFA $A = (Q, \Sigma, \delta, q_0, F)$ that decides L .

Define DFA $A' = (Q', \Sigma, \delta', q'_0, F')$, which decides $\text{Repeat}(L)$ as follows:

- $Q' = Q \times (\Sigma \cup \{\perp\}) \cup \{q_{\text{rej}}\}$, where $\perp$ indicates we are expecting the first symbol of a pair.
- Start state: $q'_0 = (q_0, \perp)$.
- Accepting states: $F' = F \times \{\perp\}$.
- Transition function δ' :

$$\begin{aligned} \delta'((q, \perp), \sigma) &= (q, \sigma), \quad \sigma \in \Sigma \\ \delta'((q, \sigma), \alpha) &= \begin{cases} (\delta(q, \sigma), \perp) & \text{if } \alpha = \sigma \\ q_{\text{rej}} & \text{if } \alpha \neq \sigma \end{cases} \\ \delta'(q_{\text{rej}}, \sigma) &= q_{\text{rej}} \quad \forall \sigma \in \Sigma \end{aligned}$$

Correctness:

A' enforces even length by interleaving jumps between $(q, \perp)$ and (q, σ) , only having $(q, \perp)$ accepting states. It also enforces equality of every symbol pair by moving first to (q, σ) and then if the next letter is not σ , transitions to q_{rej} . So any invalid "Repeat" format word is rejected.

Let $w = w_1 \dots w_n \in \Sigma^*$ and $w' = w_1 w_1 \dots w_n w_n$. I.e., $w \in L \iff w' \in \text{Repeat}(L)$.

Let $q_0, q_1, \dots, q_{n-1}, q_n$ be the run of A over w . Then, according to the transition function δ' construction, $(q_0, \perp), (q_0, w_1), (q_1, \perp), (q_1, w_2), \dots, (q_{n-1}, \perp), (q_{n-1}, w_n), (q_n, \perp)$ is the run of A' over w' . In other words, if A finishes its run over some $x \in \Sigma^*$ at state q then A' finishes its run at state $(q, \perp)$. As $q \in F \iff (q, \perp) \in F'$ by definition of F' then w is accepted by A iff w' is accepted by A' thus, $L(A') = \text{Repeat}(L)$. $\square$

Question 4

We will show that $\text{HALT} \leq_m \text{SELF} - \text{HALT}$. We saw that if L_1 is RE-hard and $L_1 \leq_m L_2$, then L_2 is RE-hard. Therefore, since we saw that HALT is RE-hard, the claim will follow.

Construction: The reduction is $f(\langle M, w \rangle) = \langle M_w \rangle$, where M_w is a Turing machine that ignores its input and simulates the run of M on w .

Correctness: When M_w takes $\langle M_w \rangle$ as input, it (ignores the input and) simulates the run of M on w , and hence it halts if and only if M halts on w . Therefore, $\langle M_w \rangle \in \text{SELF} - \text{HALT} \iff \langle M, w \rangle \in \text{HALT}$.

Computability: We saw that given a TM M and a word w , returning a TM M_w that simulates the run of a TM M on w is computable. For example, we can encode the machine M and the word w in M_w , such that M_w first writes w on the tape and then runs M on it.

□

Question 5

Assume for the sake of contradiction that $\text{SELF-HALT} \in \mathcal{R}$. This means that there exists a Turing machine H that decides SELF-HALT . That is, H halts on every input, and for any Turing machine M , H accepts $\langle M \rangle$ if and only if M halts on input $\langle M \rangle$.

We can construct from H a TM T that, on input $\langle M \rangle \in \Sigma^*$, operates as follows:

1. Runs H on input $\langle M \rangle$.
2. When H halts, if it accepted then T enters an infinite loop.
3. If H rejected, T halts.

Now, consider the behavior of T when run on input $\langle T \rangle$.

- If T halts on $\langle T \rangle$, it must be that H rejected $\langle T \rangle$, which implies

$$\langle T \rangle \notin \text{SELF-HALT} \quad \text{so} \quad \langle T, \langle T \rangle \rangle \notin \text{HALT}.$$

That is, T does not halt on input $\langle T \rangle$, a contradiction.

- Suppose $T(\langle T \rangle)$ does not halt. Then, by construction, H must have accepted $\langle T \rangle$, which implies

$$\langle T \rangle \in \text{SELF-HALT} \quad \text{so} \quad \langle T, \langle T \rangle \rangle \in \text{HALT}.$$

That is, T halts on input $\langle T \rangle$, a contradiction.

We reach a contradiction in both possible cases, hence our assumption was false.

Question 6

We show a reduction from $\overline{\text{HALT}_{\text{TM}}} \leq_m UC$. As $\overline{\text{HALT}_{\text{TM}}}$ coRE-Hard, for all $L \in \text{coRE}$ we have $L \leq_m \overline{\text{HALT}_{\text{TM}}}$ and from transitivity of reductions we have for all $L \in \text{coRE}$, $L \leq_m UC$, thus UC is coRE-Hard as required.

We consider valid inputs to be pairs of encodings - $\langle M, w \rangle$ where M is a TM and $w \in \Sigma^*$. In the construction we ignore invalid inputs which can be taken care of separately.

Construction: For a valid encoding $\langle M, w \rangle$, the reduction TM M_f returns a TM encoding $\langle M' \rangle$ where M' works as follows:

Given an input x , if $x = aa$ reject (we can encode M' to include a in the alphabet). Otherwise M' erases x , writes w and runs like M over w . If it halts then M' accepts.

Correctness: If $\langle M, w \rangle \notin \overline{\text{HALT}_{\text{TM}}}$ then M halts on w , so $L(M') = \Sigma^* \setminus \{aa\}$. We have that $x = a \in L(M')$, but $xz = aa \notin L(M')$ (with $z = a$), thus $\langle M' \rangle \notin UC$.

If $\langle M, w \rangle \in \overline{\text{HALT}_{\text{TM}}}$ then M does not halt on w , thus $L(M') = \emptyset$ and in a void manner we have that $\forall x \in L(M'), \forall z \in \Sigma^*, xz \in L(M')$, thus $\langle M' \rangle \in UC$.

Computability: We saw it is computable given a TM M and a word w to encode a TM that ignores the input and runs M over w . It is also possible to create a TM that first verifies if an input is aa (a constant word). Thus the entire reduction is computable.

□

Question 7

We will show that $\text{CLIQUE} \leq_p \text{EVEN} - \text{CLIQUE}$. We saw that if L_1 is NP-hard and $L_1 \leq_p L_2$, then L_2 is NP-hard. Therefore, since we saw that CLIQUE is NP-hard, the claim will follow.

Construction: The reduction is

$$f(\langle G, k \rangle) = \begin{cases} \langle G, k \rangle & \text{If } k \text{ is even} \\ \langle G', k+1 \rangle & \text{Otherwise (in this case } k \text{ is even),} \end{cases}$$

where if $G = (V, E)$, then $G' := (V \cup \{v'\}, E \cup \{(v, v') : v \in V\})$. That is, G' is the graph resulting from adding to G a vertex $v' \notin V$ and connecting it to all other vertices.

Correctness: Notice that in both cases the reduction returns an even number. Therefore, the parity condition always holds, and we only have to show that there is a clique of the required size in the returned graph if and only if there is a clique of size k in G .

1. If k is even, $f(\langle G, k \rangle) = \langle G, k \rangle$. The equivalence trivially holds because the reduction returns its input.
2. Otherwise, when k is odd, $f(\langle G, k \rangle) = \langle G', k+1 \rangle$. We show both directions:
 - ($\Rightarrow$) If there exists a clique of size k in G , since v' is connected to all vertices, this clique with the addition of v' is a clique of size $k+1$ in G' .
 - ($\Leftarrow$) We assume that there exists a clique of size $k+1$ in G' . If the clique contains v' , then this clique without v' is clique of size k in G . Otherwise, this is a clique of size $k+1$ in G , and in particular G has a clique of size k (by removing an arbitrary vertex from this clique).

Computability: Checking the parity of a number and adding a single vertex to a graph and connecting it to all vertices can be computed in polynomial time in the input.

□

Question 8

We will show the reduction

$$NY-L \leq_p \overline{NY-L},$$

by constructing a polynomial-time computable function f .

In the solution we consider valid inputs to be $Y \circ x$ or $N \circ x$ for all $x \in \Sigma^*$, and we will ignore invalid inputs which can be verified separately.

The reduction function works as follows:

$$f(\sigma) = \begin{cases} N \circ x & \text{if } \sigma = Y \circ x \\ Y \circ x & \text{if } \sigma = N \circ x \end{cases}$$

Correctness. We check two directions:

Case 1: If $\sigma \in NY-L$, then either

- $\sigma = Y \circ x$ with $x \in L$. Then $f(\sigma) = N \circ x$. Since $x \in L$, we have $N \circ x \notin N-L$ (because $N-L$ contains $N \circ x$ with $x \notin L$) and $N \circ x \notin Y-L$ (because it starts with N), hence $N \circ x \in \overline{NY-L}$.
- $\sigma = N \circ x$ with $x \notin L$. Then $f(\sigma) = Y \circ x$. Since $x \notin L$, we have $Y \circ x \notin Y-L$ (because $Y-L$ contains $Y \circ x$ with $x \in L$) and $Y \circ x \notin N-L$ (because it starts with Y), so $Y \circ x \in \overline{NY-L}$.

Case 2: If $\sigma \in \overline{NY-L}$, then either

- $\sigma = Y \circ x$ with $x \notin L$. Then $f(\sigma) = N \circ x$, and since $x \notin L$, we have $N \circ x \in N-L \subseteq NY-L$.
- $\sigma = N \circ x$ with $x \in L$. Then $f(\sigma) = Y \circ x$, and since $x \in L$, we have $Y \circ x \in Y-L \subseteq NY-L$.

Computability. The function f reads the first symbol of σ and modifies the prefix accordingly; the rest of the string is unchanged. This can clearly be computed in $O(|\sigma|)$ time, hence f is computable in polynomial time.

By transitivity of polynomial-time reductions, since $NY-L$ is NP-hard, we have $\overline{NY-L}$ is NP-hard.

Moreover, since $\overline{NY-L}$ is the complement of an NP language, it follows that $\overline{NY-L} \in \text{co-NP}$.

Let $k \in NP$. Then there is a polynomial-time reduction $k \leq_p \overline{NY-L}$, and by the reduction theorem, this implies $k \in \text{co-NP}$. Therefore, $NP \subseteq \text{co-NP}$.

Conversely, let $k \in \text{co-NP}$. Then $\bar{k} \in NP$, so there is a reduction $\bar{k} \leq_p \overline{NY-L}$, and applying the reduction theorem gives $\bar{k} \in \text{co-NP}$, which implies $k \in NP$. Thus, $\text{co-NP} \subseteq NP$.

Hence we conclude $NP = \text{co-NP}$.

Question 9

The claim is correct.

Since f is computable in space $O(\log(n))$ then there is a logarithmic TM M_f that performs $f(y)$ using $O(\log(n))$ space (in the working tape). Similarly, there is a logarithmic TM M_g that performs $g(x)$ in $O(\log(n))$ space.

We define a TM M that evaluates $f \circ g$ in logarithmic space by simultaneously running M_f , and M_g as follows: Given some input x with $|x| = n$, M keeps track of 2 counters in the working space.

1. Counts where the current head of M_f is (what letter of $g(x)$).
2. Counts steps of g 's printings.

In addition, the working tape contains a working space for M_f , and a working space for M_g .

Whenever M_f wants to move the reading head left over the "input" $g(x)$, we reduce counter 1 by 1. Whenever M_f wants to move the reading head right over the input $g(x)$, we increase counter 1 by 1. Then we reset counter 2, and perform $M_g(x)$. Whenever $M_g(x)$ prints a letter we increase counter 2 by 1 till it is equal to counter 1. When that happens we also write this letter on the working tape, and let M_f perform with the given input letter.

The correctness is trivial, M_f receives the correct input letter of $g(x)$ whenever required and can perform its steps with regards to it.

Complexity:

We essentially have to keep 4 things on the working tape:

1. Working tape space for M_f .
2. Working tape space for M_g .
3. Counter for M_f which letter $g(x)$ we are currently on.
4. Counter for M_g to print the required letter that M_f expects.

First, since M_g is computable and runs in $O(\log(n))$ space, it necessarily halts. As such, no configuration repeats (or there would be a loop). There are

$$|Q| \times \underset{\text{reading head positions for } x}{n} \times \underset{\text{working head positions}}{O(\log(n))} \times \underset{\text{content options of the working tape}}{|\Gamma|^{O(\log(n))}} = \underset{\text{for some constant } d}{O(n^d)}$$

configurations. Thus, $|g(x)| = O(n^d)$ (as we can print once for each configuration).

Now we show that indeed each item in the working tape requires logarithmic space.

Since $|g(x)| = O(n^d) \implies |g(x)| \leq c \cdot n^d$ for some constant c , then both counters require $\log(c \cdot n^d) = \log(c) + d \log(n) = O(\log(n))$ space.

M_g requires $O(\log(n))$ as it is a logarithmic space function.

M_f requires $O(\log(c \cdot n^d)) = O(\log(c) + d \log(n)) = O(\log(n))$ space.

Each component on the working tape requires $O(\log(n))$ space throughout the entire run, thus the space required in total is indeed logarithmic. $\square$